

ExempleCallable (v4)

Fabrice.Kordon@lip6.fr



Objectif de cette version

2

Liste de tâches

- Insertion dans un pool
- Attente et récupération des résultats
 - ▶ Toujours une somme sur la liste de résultats
- Mais lancement en bloc des travaux via une liste

La classe Calcule
ne change pas

Vite mon
aspirine!



La «classe principale»

3

```
public class ExempleCallable4 {
    static int nbThreads;
    static int taillePool;

    public static void main(String []args) {
        int somme = 0;

        try {
            nbThreads = Integer.parseInt(args[0]);
            taillePool = Integer.parseInt(args[0]);
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.err.println ("Arguments requis, nombre de threads, taille du pool.");
            return;
        }

        ExecutorService executor = Executors.newFixedThreadPool(taillePool);
        List<Calcule> lesTaches = new ArrayList<Calcule>();

        System.out.println("main : je construit la liste de calculs");
        for (int i = 1; i <= nbThreads ; i++) {
            lesTaches.add(new Calcule("c"+i));
        }
    }
}
```


La «classe principale»

3

```
System.out.println("main : je lance les calculs");
try {
    // lancement en bloc. Peut lever InterruptedException, NullPointerException
    // ou RejectedExecutionException
    List<Future<Integer>> lesResultats = executor.invokeAll(lesTaches);
    // Synchronisation implicite (à la cobegin/coend)
    System.out.println("main : J'ai eu les résultats");
    for (Future<Integer> res : lesResultats) { // parcourir une collection
        try{
            int x = res.get();
            System.out.println("main : " + x + " reçu à " + new Date());
            somme = somme + x;
        }
        catch (Exception e) {
            e.printStackTrace();
        }
    }
}
catch (InterruptedException e) {
    System.out.println(e + " ne devrait pas arriver");
}
System.out.println("main : j'ai calculé " + somme);
executor.shutdown();
}
```


Exécution

4

```
$ java ExempleCallable4 12 3
main : je construit la liste de calculs
main : je lance les calculs
c2 rend 131
c10 rend 176
c7 rend 230
c6 rend 280
c11 rend 332
c5 rend 337
c8 rend 392
c12 rend 505
c9 rend 516
c1 rend 607
c3 rend 659
c4 rend 806
main : J'ai eu les résultats
main : 607 reçu à Wed Oct 05 11:00:03 CEST 2016
main : 131 reçu à Wed Oct 05 11:00:03 CEST 2016
main : 659 reçu à Wed Oct 05 11:00:03 CEST 2016
main : 806 reçu à Wed Oct 05 11:00:03 CEST 2016
main : 337 reçu à Wed Oct 05 11:00:03 CEST 2016
main : 280 reçu à Wed Oct 05 11:00:03 CEST 2016
main : 230 reçu à Wed Oct 05 11:00:03 CEST 2016
main : 392 reçu à Wed Oct 05 11:00:03 CEST 2016
main : 516 reçu à Wed Oct 05 11:00:03 CEST 2016
main : 176 reçu à Wed Oct 05 11:00:03 CEST 2016
main : 332 reçu à Wed Oct 05 11:00:03 CEST 2016
main : 505 reçu à Wed Oct 05 11:00:03 CEST 2016
main : j'ai calculé 4971
```



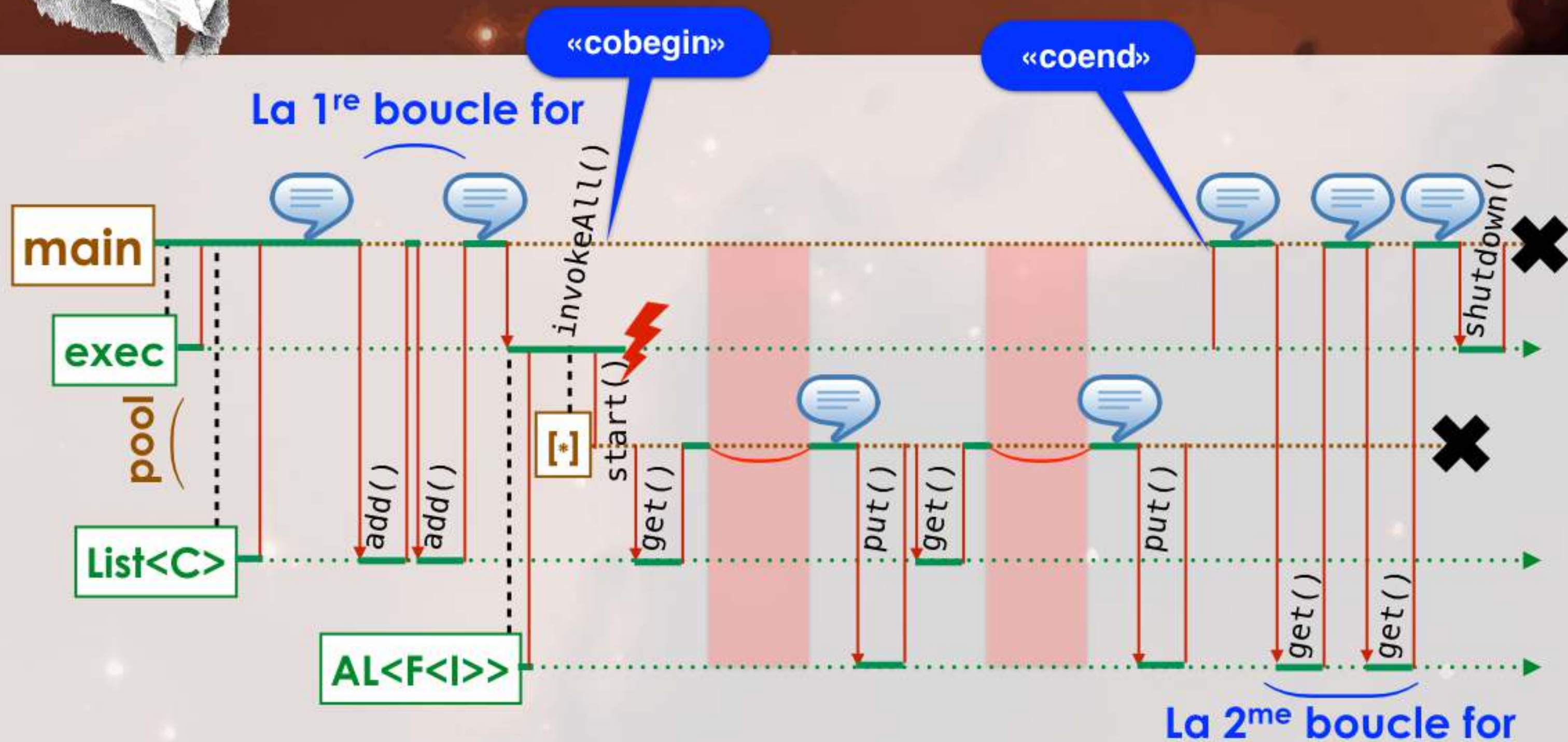
Observation

Attente avant l'affichage
«j'ai eu les résultats»



Encore un chronogramme...

5



```
$ java ExempleCallable4 2 1
main : je construit la liste de calculs
main : je lance les calculs
c2 rend 12
main : j'ai eu les résultats
c1 rend 472
main : 472 reçu à Fri Oct 30 18:53:55 CET 2015
main : 12 reçu à Fri Oct 30 18:53:55 CET 2015
main : j'ai calculé 484
```